

SmartTaps: Voice-Controlled Interaction for Smartphones (Deliverable 3)

1st Michael Calderin

Department of Engineering Education

University of Florida

Gainesville, FL, USA

michaelcalderin@ufl.edu

Abstract—SmartTaps is an innovative and advanced voice-controlled software framework meticulously designed to enable users to interact with their smartphones in a completely hands-free manner by converting spoken commands into precise and accurate taps on the screen. This system effectively bridges the gap between natural human language and the structured graphical user interfaces (GUIs) commonly found in mobile devices, allowing for seamless navigation and operation without the need for physical touch. By integrating state-of-the-art technologies such as real-time speech recognition, sophisticated vision-language understanding models, and automated device control through the Android Debug Bridge (ADB), SmartTaps represents a significant step forward in enhancing user accessibility and convenience. This report provides a detailed summary of the improved implementation of the system as of November 23, 2025, including an in-depth update on the new architecture, the comprehensive model training process, evaluation results from testing, comparison to the previous iteration, and a thoughtful discussion on the various challenges encountered during development, as well as important ethical considerations surrounding the deployment of voice-driven automation technologies. The framework not only facilitates everyday tasks but also holds potential for assisting individuals with disabilities, thereby promoting inclusivity in technology use. For more details, visit the project repository: <https://github.com/michaelcalderin/smart-taps>.

A. PROJECT SUMMARY

The goal of this project is to build a deep learning system that can automatically detect and localize user interface (UI) elements on mobile phone screens. This capability is important for applications such as accessibility improvements and digital screen understanding. In my previous approach, I used CLIP, a model trained for image-text matching, and attempted to adapt it for object detection by adding a custom regression head to predict bounding box coordinates. However, CLIP is not designed for spatial localization, and the results were poor: bounding boxes were often random and failed to consistently lock onto UI elements.

To address these limitations, I transitioned to OWL-ViT (Open-Vocabulary Object Detection with Vision Transformers), a transformer-based model specifically designed for open-vocabulary object detection. OWL-ViT is trained to predict bounding boxes for objects described by text queries, making it inherently suited for tasks where the goal is to find specific objects in an image based on their textual description. Unlike CLIP, OWL-ViT's architecture includes both a classification head (to identify which image patch contains the queried

object) and a regression head (to predict the precise bounding box coordinates). This means I no longer need to design or train a separate regression module, as the model natively supports end-to-end fine-tuning for both classification and localization.

Fine-tuning OWL-ViT on my annotated dataset involved preparing each sample as a screen image, a text label describing the UI element, and normalized bounding box coordinates. Images and text queries are processed using the OWL-ViT processor, which handles resizing, tokenization, and batching. The model is trained to maximize the likelihood of correctly classifying the patch containing the UI element and accurately regressing the bounding box coordinates. Loss functions include focal loss for classification and smooth L1 loss for box regression, with Hungarian matching used to assign predictions to ground truth boxes.

After switching to OWL-ViT and fine-tuning on my dataset, performance improved noticeably. The predicted bounding boxes are less random and more likely to lock onto actual UI elements, even if the model sometimes selects the wrong element. The model's built-in regression head produces more coherent box predictions, and the overall detection process is more stable compared to the CLIP-based approach. While the results are still not perfect (it makes mistakes much too often for practical use) the transition to OWL-ViT has made the system more reliable and demonstrates the importance of using a model architecture that is well-suited for the task.

These refinements matter because they have led to more meaningful predictions and a more streamlined training process. By leveraging a model designed for object detection, I have eliminated the need for custom regression heads and complex post-processing, allowing for more direct and effective learning. OWL-ViT also accepts larger input images (such as 768x768), unlike CLIP which was limited to 224x224, enabling the model to capture more detail and context from each screen. This flexibility is especially important for UI analysis, where elements can be small or densely packed. These improvements are crucial for achieving the project's goals and lay the groundwork for future enhancements. Although results are still unfavorable, this architecture shows more promise given the right resources and time.

B. UPDATED SYSTEM ARCHITECTURE AND PIPELINE

The updated system architecture is shown in Figure 1. The overall pipeline remains conceptually consistent: a user speaks a command, which is transcribed by a speech-to-text model (Whisper), and simultaneously, a screenshot of the device is captured. Both the transcribed text and the screenshot are then processed and jointly fed into a vision-language model, which predicts the bounding box corresponding to the UI element mentioned in the command. The bounding box is initially represented in normalized coordinates relative to the image dimensions and is subsequently converted into device-specific coordinates to allow the system to emulate a tap on the target location accurately. This ensures that the interaction remains precise across devices of different resolutions or aspect ratios.

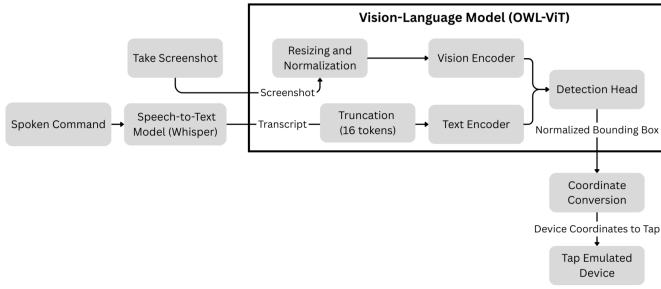


Fig. 1. Updated architecture using OWL-ViT instead of CLIP as the vision-language model.

In this updated design, the vision-language model has been changed from CLIP to OWL-ViT, following the methodology presented in [1]. OWL-ViT extends the standard Vision Transformer (ViT) architecture by introducing a detection head that combines classification and regression directly on patch-level embeddings. Unlike CLIP, which produces a single global image embedding and is optimized for image-text similarity, OWL-ViT treats each patch token as a potential candidate for object localization. Each patch embedding passes through a classification head, which scores the likelihood of the patch containing the target object (based on the text embedding), and a regression head, which predicts the bounding box coordinates (x, y, w, h) in normalized form relative to the processed image’s dimensions. The classification weights are dynamically derived from the text embedding produced by the text encoder, enabling open-vocabulary object detection, which is critical for processing arbitrary UI commands.

The text encoder in OWL-ViT, which is typically a transformer identical to CLIP’s text encoder, processes tokenized input commands into a dense semantic embedding. This embedding is L2-normalized and then used as the classification vector for each vision patch, allowing the model to score every patch according to its similarity to the command. Unlike the vision encoder, which outputs spatially distributed embeddings, the text encoder produces a single vector representation of the entire command sequence. The text input is tokenized, truncated to a maximum of sixteen tokens, and padded as necessary. Limiting the token length focuses the model on

the most relevant part of the command, avoiding distraction from extraneous words while maintaining compatibility with the encoder’s fixed input size.

In terms of fine-tuning for this deliverable, the model was trained end-to-end using a combination of a focal sigmoid cross-entropy loss for classification and smooth L1 loss for the bounding boxes. To be consistent with OWL-ViT’s initial training, the Hungarian bipartite matching algorithm was applied to optimally pair predicted boxes with ground truth boxes. This matching ensures that each prediction is compared against the most relevant ground truth, reducing ambiguity in training and improving localization accuracy. For this project, there was only one ground truth so all predicted bounding boxes were compared to that one box.

In the fine-tuning process, the detection head was fully unfrozen, enabling it to adapt rapidly to the target UI element localization task. Additionally, the last two layers of the text encoder and the last four layers of the vision encoder were unfrozen. Previously, with CLIP, four layers of both encoders were unfrozen, but in the case of OWL-ViT, less layers were unfrozen. Furthermore, differential learning rates are applied: the detection head receives a higher learning rate to adapt quickly, while the fine-tuned layers of the text and vision encoders are updated with smaller learning rates. This approach preserves the pre-trained cross-modal representations while still allowing the model to adjust to the specific task. The decision to unfreeze part of the text encoder is informed by the OWL-ViT paper, which reported poor performance when the text encoder was entirely frozen, suggesting that slight adaptation of textual representations improves the alignment between textual queries and spatial vision features. Compared to previous attempts, less layers were unfrozen to reduce computational load.

The adoption of OWL-ViT introduces several advantages. CLIP’s global image embeddings were not spatially grounded, which often led to inconsistent or inaccurate bounding boxes when attempting to localize UI elements. OWL-ViT, by operating on patch-level embeddings and incorporating a built-in detection head, produces bounding boxes that are both semantically meaningful and spatially precise. Additionally, the increase in input image resolution to 768×768 pixels, compared to 224×224 used with CLIP, allows finer-grained spatial information, further improving the accuracy of the localization. The combination of normalized bounding boxes, Hungarian matching, and differential learning rates ensures that the model generalizes well to different commands and device screenshots, maintaining higher precision in predicted tap locations.

The integration of OWL-ViT with carefully designed fine-tuning strategies, including selective unfreezing of the text and vision encoders, use of differential learning rates, and higher-resolution image inputs, enhances the system’s ability to localize UI elements in an open-vocabulary setting. This design effectively leverages the model’s pre-trained semantic and spatial understanding while adapting it to the task of executing spoken commands on Android devices with precise

interactions.

C. REFINEMENTS MADE SINCE DELIVERABLE 2

Since Deliverable 2, several refinements have been made to improve system performance, robustness, and usability, particularly with the transition from CLIP to OWL-ViT.

The preprocessing pipeline has been significantly improved. The text input to the model is truncated to a maximum of sixteen tokens before being fed into the text encoder. This ensures that the input fits within the model’s expected token length and avoids issues with overly long sequences, while retaining the core command content for processing. Image inputs are consistently resized and normalized to meet OWL-ViT’s requirements. Rather than padding images to maintain aspect ratio, screenshots are squashed vertically to fit the model’s input size of 768×768 pixels. This approach, combined with OWL-ViT’s support for larger input images, reduces distortion and preserves visual clarity. As a result, text and UI elements in the images are sharper and more readable, in contrast to the previous CLIP-based approach, where small input sizes and padding often led to blurry, low-resolution images.

The features and labels have also been refined. The label format now uses normalized center coordinates and width/height, aligning with the expectations of OWL-ViT’s detection head. This tighter coupling between features and model architecture improves training efficiency and ensures that predictions are more directly compatible with the network’s outputs.

In terms of hyperparameter tuning, learning rates for the detection head, vision encoder, and text encoder are still set differentially, allowing each component to adapt appropriately. The detection head is updated more aggressively, while the encoders are fine-tuned with lower rates to preserve pre-trained representations. The batch size has been adjusted to balance stability and hardware constraints. Loss weights for classification and regression have been balanced carefully to optimize both tasks simultaneously.

The training loop has been enhanced with several convergence improvements. A learning rate warmup phase is now applied at the beginning of training, followed by cosine annealing, which improves convergence and prevents sudden large updates. These modifications were frequent recommendations when reviewing similar models and projects.

Model stability has also been improved. Gradient clipping is now applied to prevent exploding gradients, which can destabilize training. Hungarian bipartite matching is used to assign predictions to ground truth boxes robustly, ensuring that each object is paired optimally and that unmatched predictions are treated as background. For bounding box regression, Smooth L1 loss has replaced CIOU loss. This change focuses training on distance errors rather than strict geometrical constraints, which is particularly effective with limited training data, as it produces more stable gradients early in training. OWL-ViT’s detection head produces bounding boxes that tightly fit UI element shapes from the start, unlike the CLIP-based system, which often predicted large, inaccurate boxes covering most of the screen.

Training has been particularly challenging due to resource constraints and dataset limitations. To evaluate what works best, training has been conducted on a small subset of the data, which makes experimentation slow because each training run takes a while to finish. Even now, it remains unclear how many layers should be unfrozen or which hyperparameters are optimal. Resources are insufficient to train on the large-scale datasets that OWL-ViT was originally trained on. Additionally, the dataset itself presents difficulties. Textual descriptions of UI elements are often overly literal and do not reflect how a typical user would describe an element. Some pages contain multiple elements that match the same description, but the textual input does not provide a way to differentiate them, for example by specifying general positions, creating inherent ambiguity for the model.

Collectively, these refinements, including improved preprocessing, tighter feature-label alignment, careful hyperparameter tuning, a more robust training loop, and enhanced model stability, have led to substantial improvements in detection accuracy, convergence speed, and overall system reliability, despite the significant challenges posed by limited data and compute resources.

D. INTERFACE USABILITY AND IMPROVEMENTS

Although no direct modifications were made to the interface layout itself, significant improvements in the vision-language model have led to implicit enhancements in the user experience. The system now uses an emulated Google Pixel device, a widely recognized Android phone, to demonstrate interactions. This choice ensures compatibility with the dataset (only used Android) and provides a familiar interface for users to observe system behavior.

Figure 2 shows the emulated device displaying a web browser search screen with a blue clock UI element. When the spoken command “blue clock” is issued, the system first transcribes the speech using a speech-to-text model, then processes the screenshot through the vision-language model to predict the appropriate tap location. As shown in Figure 3, the device successfully launches the clock app after tapping the predicted bounding box. This demonstrates that the system can accurately map natural language commands to UI interactions on a real device, supporting that the concept for this project is feasible, especially for large companies that have more resources for training.

To further illustrate model performance, Figure 4 displays the original phone screen along with the top three predicted bounding boxes for the tap location. The top prediction is placed on the “clock” text below the blue clock icon, indicating that the model has developed a partial understanding of textual cues on the screen. This is a notable improvement over previous CLIP-based versions, where predictions were often random or covered large portions of the screen, resulting in poor usability. However, the ideal behavior would have the top prediction directly on the blue clock icon itself, highlighting areas for further refinement in bounding box precision and text-to-object alignment.

A potential future improvement is to have the system continuously listen for spoken input rather than requiring a user to manually press a button to record commands. This feature has not yet been implemented, as the current focus has been on improving the vision-language model. While the model is performing better than in previous versions, it is still not accurate or robust enough for real-world continuous use. In fact, due to time and resource constraints, continuous listening may ultimately not be implemented. At present, the system functions primarily as a proof of concept, demonstrating the feasibility of mapping spoken commands to UI interactions with improved accuracy and responsiveness.

Overall, these results demonstrate that while the interface layout remains unchanged, advancements in the underlying vision-language model have enhanced the accuracy, responsiveness, and visual meaningfulness of interactions. Users now experience a more reliable mapping between spoken commands and UI responses, providing a smoother and more intuitive interface even without explicit design changes. The combination of improved predictions and familiar device emulation contributes to a more polished and usable proof-of-concept system.

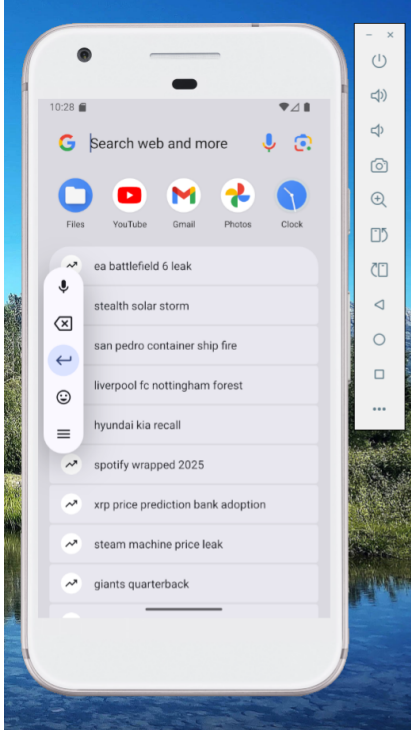


Fig. 2. Emulated Google Pixel device showing a web browser with a blue clock UI element.

E. EXTENDED EVALUATION AND UPDATED RESULTS

To assess progress in UI element detection, the performance of the previous CLIP-based model was compared with the current OWL-ViT approach. This comparison highlights both quantitative improvements and qualitative changes in bounding box localization.



Fig. 3. Device after the "blue clock" command, showing the clock app launched.

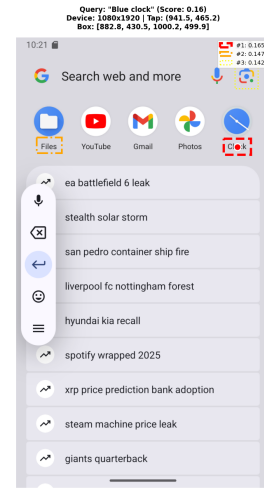


Fig. 4. Original phone screen with top three predicted bounding boxes. The top prediction is on the "clock" text below the icon.

The CLIP-based model was trained for five epochs on a subset of 10,000 samples. Due to the model's limitations, including smaller image inputs, the lack of a dedicated detection head, and an architecture not designed for object localization, bounding box predictions were largely unreliable. Quantitatively, the model achieved mean IoU scores of 0.052 on the training set, 0.057 on the validation set, and 0.057 on the held-out test set. The corresponding loss values were 1.16, 1.21, and 1.21. These metrics reflected the model's inability to accurately localize UI elements. Qualitative examples demonstrated that predicted bounding boxes were often large, misplaced, or covered irrelevant portions of the interface.

In contrast, the OWL-ViT model was fine-tuned for one epoch on the same subset of 10,000 examples. Given the short training schedule, meaningful loss curves could not be generated, but the best model saved after epoch one achieved a best validation IoU of 0.1935. On the held-out test set, OWL-ViT reached a test loss of 0.0020, a mean test IoU of 0.1910, and a median test IoU of 0.0000. While these metrics indicate that the model still struggles with consistently high IoU scores, they represent a substantial improvement over the CLIP baseline. The model is now capable of producing bounding boxes that partially align with actual UI elements.

Qualitative analysis supports these quantitative results. Figure 5 shows four sample screens with ground truth bounding boxes and the top predicted bounding box for each. In these examples, predicted boxes are usually tightly wrapped around UI elements, which indicates that the model has learned to localize objects precisely. However, in most cases, the boxes are placed on the wrong element. Out of the four samples, only one prediction is correct, corresponding to a small briefcase icon. This correct detection demonstrates the model’s ability to identify small icons, highlighting its potential even with limited data and training. The score associated with each predicted box represents the model’s confidence that the box contains the UI element described by the text query.

These results highlight several important insights about the system’s learning and reliability. The dedicated detection head in OWL-ViT, combined with higher-resolution images, significantly improves the model’s ability to propose plausible bounding boxes. At the same time, the small dataset and short training schedule limit generalization, resulting in many predictions that do not overlap with ground truth. Despite these limitations, qualitative gains suggest the model has potential. Predictions are tighter and more structured than with CLIP, and the correct detection of the small briefcase icon shows that the model can localize fine-grained elements when it is trained sufficiently.

Future work will focus on longer training schedules, expanded datasets, and further architectural refinements to improve localization accuracy. Given current resource constraints, the system primarily serves as a proof of concept, demonstrating that OWL-ViT can generate structured and interpretable predictions even with limited data.

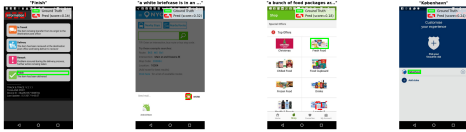


Fig. 5. Four sample screens showing ground truth bounding boxes and top predicted bounding boxes. Predicted boxes are usually tightly wrapped around UI elements, but in most cases they are placed on the wrong element. Only the small briefcase icon is correctly predicted. The score indicates the model’s confidence that the predicted box contains the target UI element described by the text query.

F. RESPONSIBLE AI REFLECTION

Throughout the development and testing of this system, several responsible AI considerations surfaced, particularly around fairness, bias, privacy, and user trust. The dataset used for training is composed of annotated screenshots from a relatively small set of mobile applications and device types, which naturally introduces bias. The limited visual diversity of UI styles may mean the model may perform better on interfaces that resemble the training samples while struggling with unfamiliar layouts, different color schemes, or app categories that are underrepresented. No formal auditing for dataset imbalance or representational bias was conducted due

to the project’s tight timeline, so any performance gaps are still largely unknown.

From a privacy standpoint, all data used during development was either synthetic or publicly available and did not contain personal information, which kept the immediate risk low. However, privacy becomes more complicated when thinking about real deployment. OWL-ViT is significantly larger and more computationally demanding than the earlier CLIP-based model, and running it efficiently on a mobile device may not be realistic. This raises the possibility of relying on cloud-based inference, which would require uploading screenshots of a user’s screen to a remote server. Even if processed securely, this introduces serious privacy concerns because the system would have temporary access to potentially sensitive visual information. Balancing computational requirements with privacy is therefore a key challenge in moving beyond a proof of concept.

User trust also plays a role, since the model’s predictions are still inconsistent and may fail on UI layouts that differ from the training distribution. To avoid misleading users, this project is emphasized as a proof of concept. This helps set appropriate expectations for reliability. Looking ahead, improvements will focus primarily on refining the vision-language model to improve detection accuracy and gain a broader understanding of UI layouts. Broader fairness audits and more robust privacy safeguards remain outside the scope of this project, but they would be essential considerations for any real-world deployment. Several additional features would be required for production such as a confirmation message before tapping to prevent wrongful execution.

REFERENCES

- [1] M. Minderer, A. Gritsenko, A. Stone, M. Neumann, D. Weissenborn, A. Dosovitskiy, A. Mahendran, A. Arnab, M. Dehghani, Z. Shen, X. Wang, X. Zhai, T. Kipf, and N. Houlsby, “Simple open-vocabulary object detection with vision transformers,” *arXiv preprint arXiv:2205.06230v2*, 2022. [Online]. Available: <https://doi.org/10.48550/arXiv.2205.06230>